

내 손으로 조작하며 배우는 5일 강화학습

260202 1일차: 강화학습의 기본 재료

내 손으로 조작하며 배우는 5일 강화학습

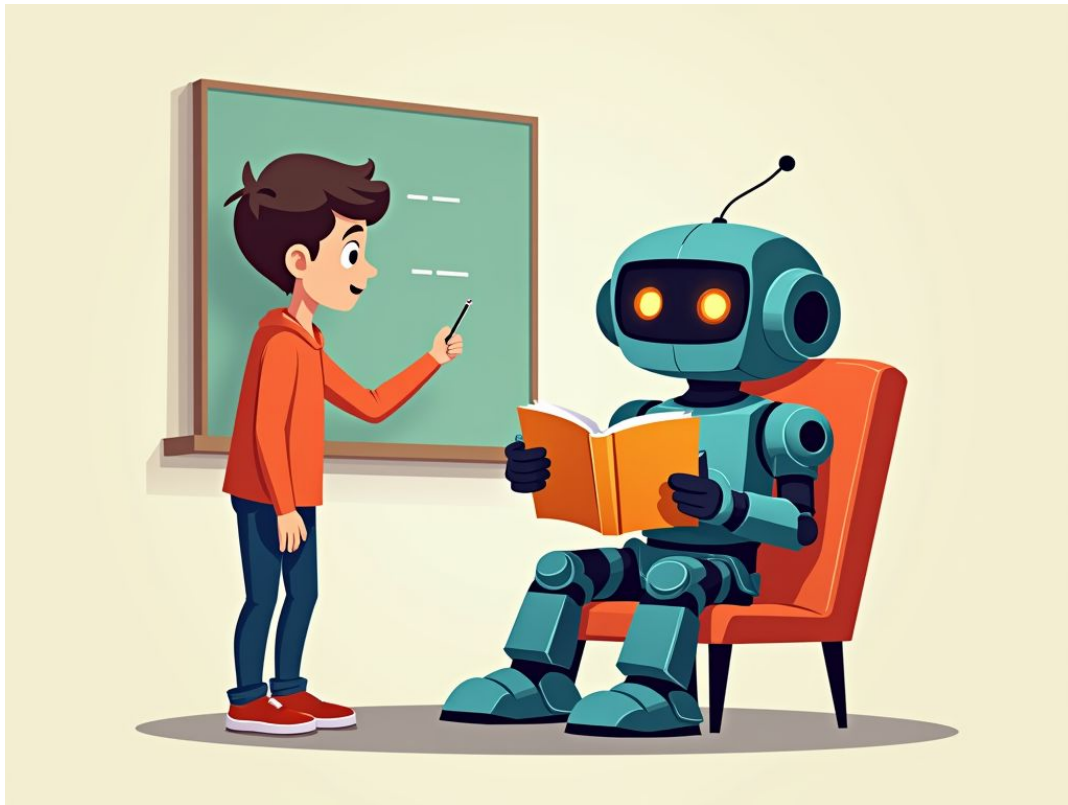
- 수강 대상:

- 파이썬 기초는 알지만, 딥러닝이나 강화학습에는 익숙하지 않은 사람들을 타겟으로 합니다.

- 최종 목표:

- 바이브 코딩을 통해 학습 과정을 그래프로 시각화하고, 하이퍼파라미터를 슬라이더로 조절하며, 에이전트가 움직이는 모습을 웹브라우저에서 볼 수 있는 'AI 엘리베이터 관제 시스템'을 만드는 것입니다.

기계 학습 (Machine Learning)



머신러닝 학습 방법 비교

- 지도학습: 정답이 있는 데이터로 학습
- 비지도학습: 정답 없이 패턴 발견
- 강화학습: 보상을 통해 시행착오로 학습

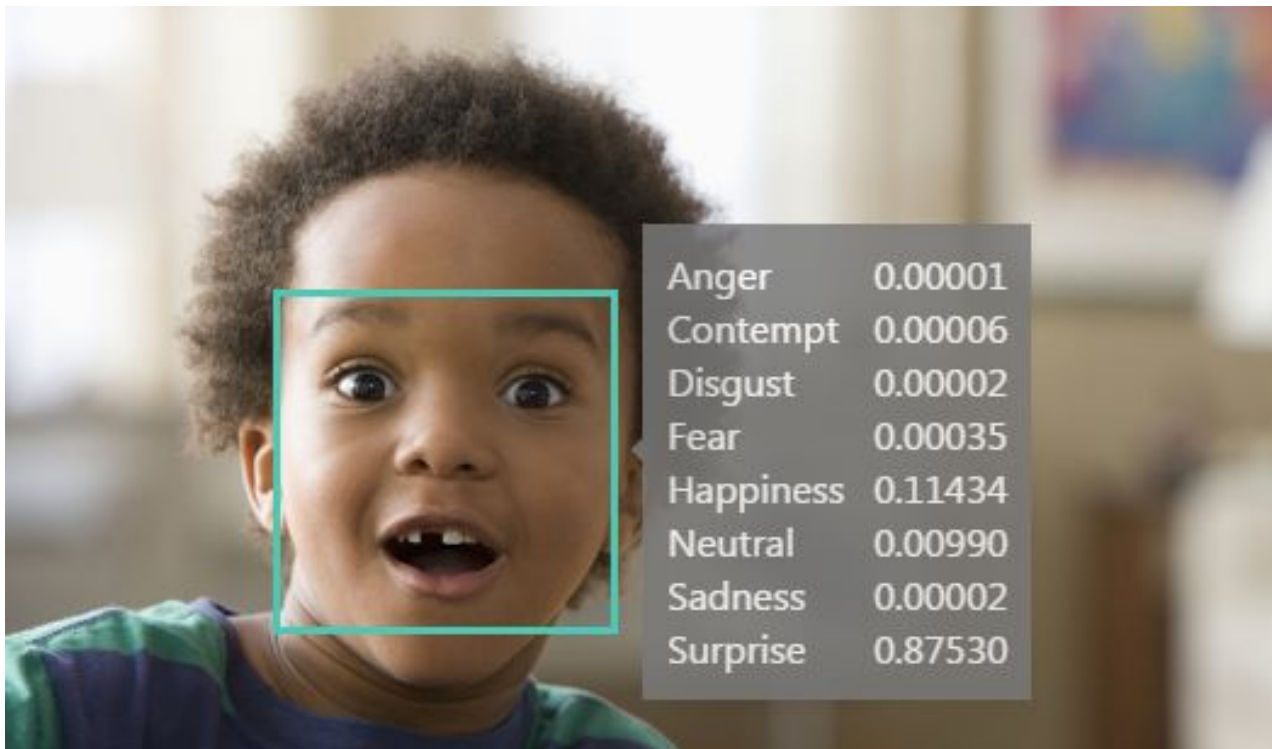
지도 학습 (Supervised Learning)



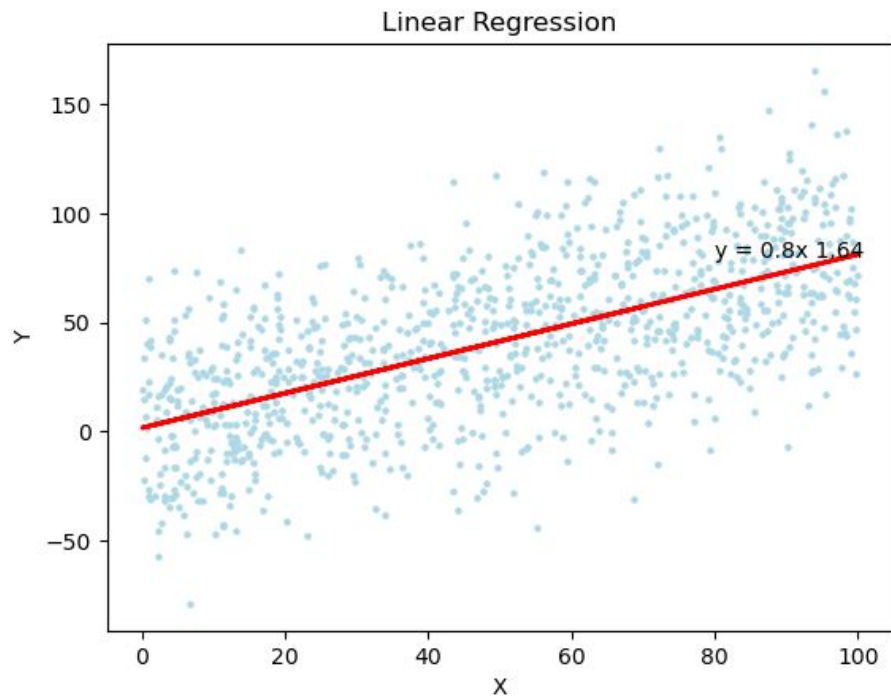
지도 학습 (Supervised Learning)

- 데이터-레이블을 사용하여 모델을 학습
- 명확한 정답을 모델에 제공
- 새로운 데이터에 대해 예측을 하는 것
 - 분류(Classification): 데이터가 어느 카테고리에 속하는지를 예측
 - 회귀(Regression): 연속적인 값을 예측
 - 이미지 분류, 스팸 탐지 등에 쓰임

지도 학습 (Supervised Learning)



지도 학습 (Supervised Learning)



비지도 학습 (Unsupervised Learning)

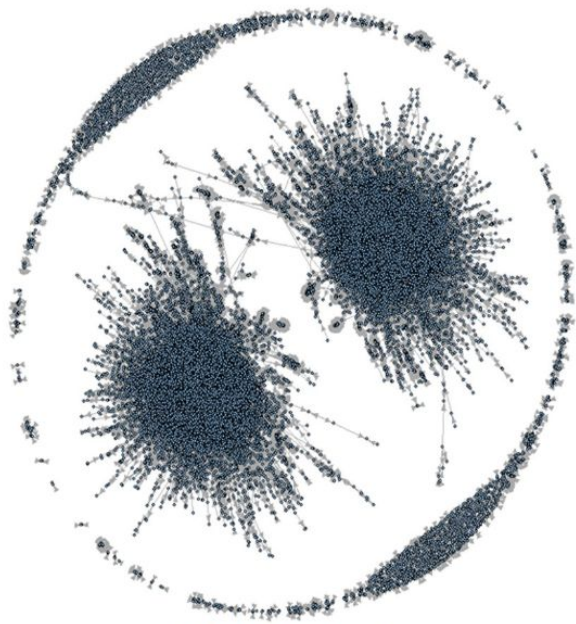


비지도 학습 (Unsupervised Learning)

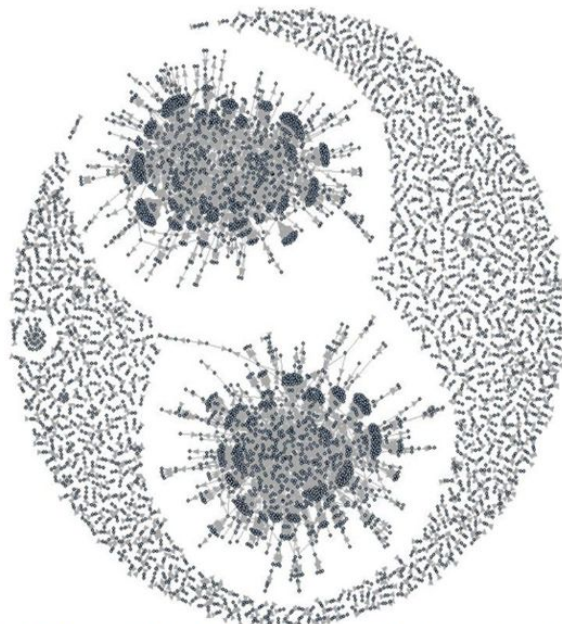
- 데이터만을 사용하여 데이터의 구조나 패턴을 발견
- 데이터의 숨겨진 특징을 탐색하거나 그룹화
 - 군집화(Clustering): 유사한 특성을 가진 데이터 포인트를 그룹화
 - 차원 축소(Dimensionality Reduction): 데이터의 특성 수를 줄여서 분석
 - 고객 세분화, 이상 탐지

비지도 학습 (Unsupervised Learning)

http://ndcreplay.nexon.com/NDC2017/sessions/NDC2017_0013.html



전체 거래 네트워크



대가 없는 게임 머니 및 아이템 이동 네트워크

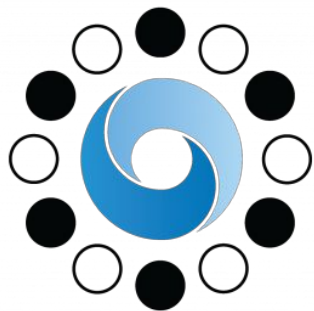
비지도 학습 (Unsupervised Learning)

<https://www.welfare24.net/ab-3101-1905>



강화 학습 (Reinforcement Learning)

- 에이전트가 환경과 상호작용하며 보상을 최대화
 - 알파고

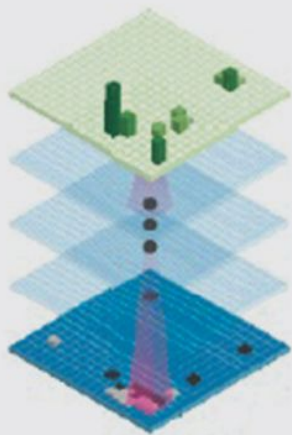


AlphaGo

강화 학습 - 알파고

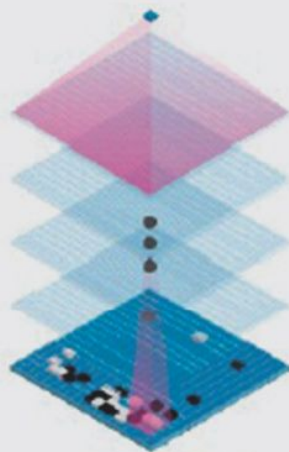
알파고의 정책망과 가치망 구조

정책망



입력된 기보를 바탕으로
특정 상황에 전문가들이 많이 두는
수로 탐색의 폭을 좁혀주는 것

가치망



걸러진 수 가운데 현재 대국
상황에서 승산이 높은 수를 예측.
형세를 판단해주는 것

강화 학습 실패 사례

<https://drive.google.com/file/d/1Xsg6flSXd1cO6NxSlKo2jvLGe3ImiJjO/view>

https://drive.google.com/file/d/1u-fwD3iiQH5Q_cYn4T_ZYzEd1-PKtLFW/view



상태 → 행동 → 보상 학습 순환

- 상태(State)

- 에이전트가 현재 처한 상황을 나타내는 정보의 집합
- 에이전트가 의사결정을 내릴 때 사용하는 환경에 대한 관찰 정보

- 행동(Action)

- 에이전트가 특정 상태에서 선택할 수 있는 가능한 모든 행위
- 에이전트의 행동 선택에 따라 환경이 변화하고 새로운 상태로 이동

- 보상(Reward)

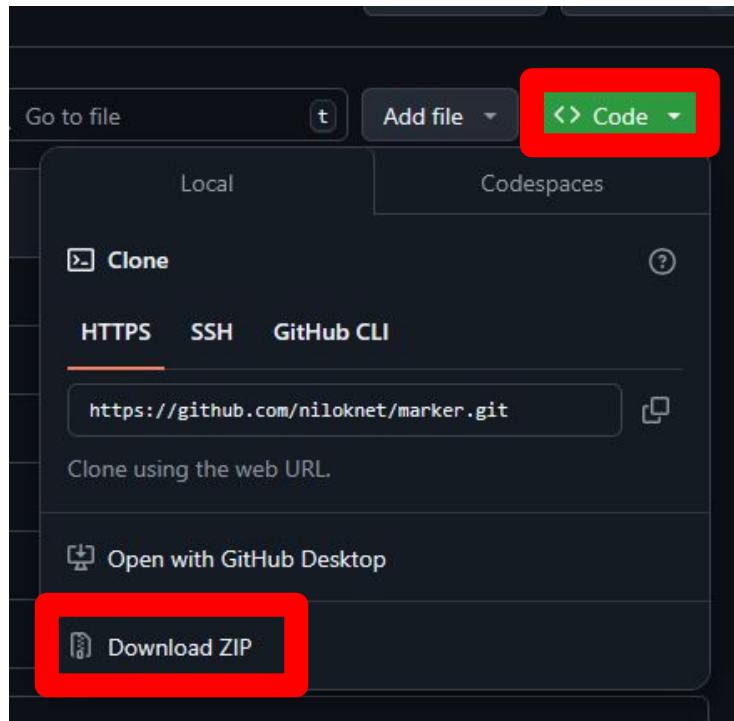
- 에이전트가 특정 행동을 취한 후 환경으로부터 받는 피드백
- 행동의 좋고 나쁨을 수치화한 평가 지표

Gymnasium 소개

- 파이썬 기반 강화학습 프레임워크
- 표준 강화학습 환경 (행동, 보상, 상태) 제공
- 다양한 예시 문제 포함
- <https://gymnasium.farama.org/index.html>

Gymnasium 환경 확인하기

<https://github.com/niloknet/marker/tree/2026-02>



Gymnasium 환경 확인하기

<https://code.visualstudio.com/download>

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows

Windows 10, 11

User Installer	x64	Arm64
System Installer	x64	Arm64
.zip	x64	Arm64
CLI	x64	Arm64



↓ .deb

Debian, Ubuntu

.deb	x64	Arm32	Arm64
.rpm	x64	Arm32	Arm64
.tar.gz	x64	Arm32	Arm64
Snap	Snap Store		
CLI	x64	Arm32	Arm64

↓ .rpm

Red Hat, Fedora, SUSE

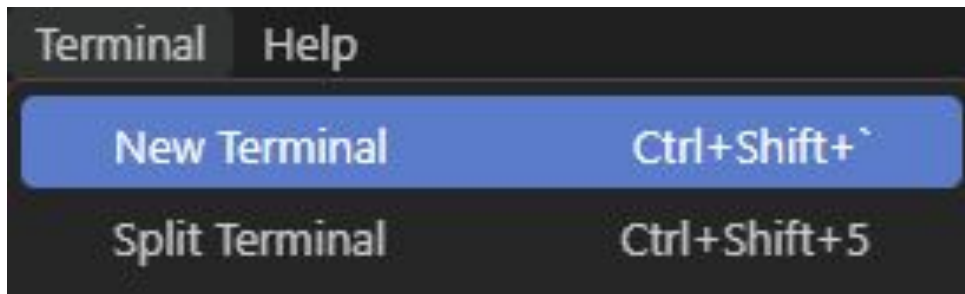


↓ Mac

macOS 11.0+

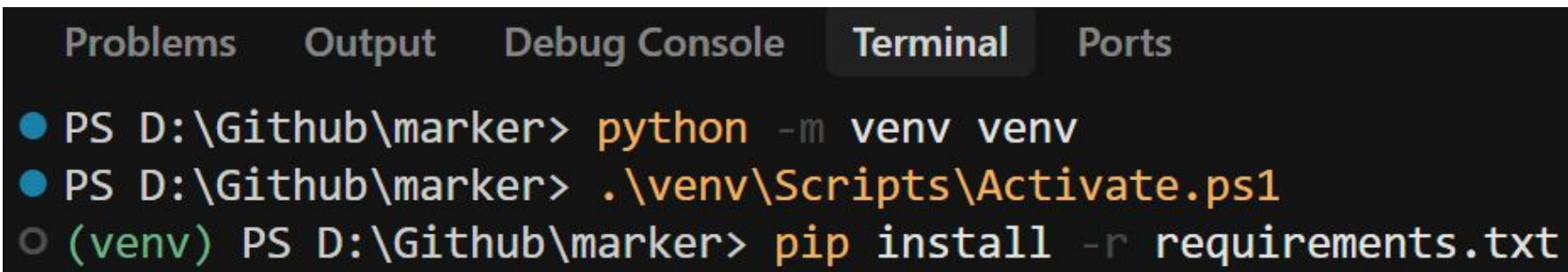
.zip	Intel chip	Apple silicon	Universal
CLI	Intel chip	Apple silicon	

Gymnasium 환경 확인하기



Gymnasium 환경 확인하기

1. `python -m venv venv`
2. `.\venv\Scripts\Activate.ps1`
3. `pip install -r requirements.txt`



The screenshot shows a VS Code interface with a terminal window open. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (which is active), and 'Ports'. The terminal output shows three commands being executed in a PowerShell prompt at the path D:\Github\marker. The first command creates a virtual environment named 'venv'. The second command activates the virtual environment. The third command installs the dependencies from a file named 'requirements.txt'. The prompt changes from 'PS' to '(venv) PS' after the second command.

```
Problems  Output  Debug Console  Terminal  Ports
● PS D:\Github\marker> python -m venv venv
● PS D:\Github\marker> .\venv\Scripts\Activate.ps1
○ (venv) PS D:\Github\marker> pip install -r requirements.txt
```

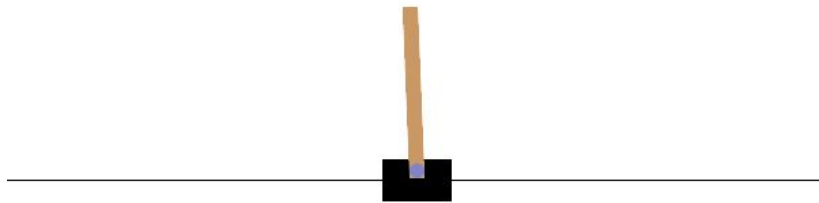
Gymnasium 환경 확인하기

```
python .\day1\01_ml_paradigms.py
```

- https://scikit-learn.org/stable/datasets/toy_dataset.html#iris-dataset
- https://scikit-learn.org/stable/datasets/toy_dataset.html#wine-dataset

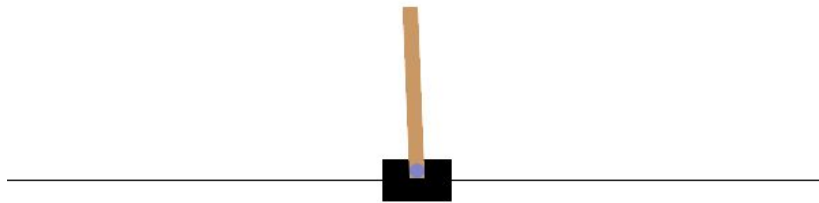
CartPole 환경

- 수레 위에서 막대 균형 잡기



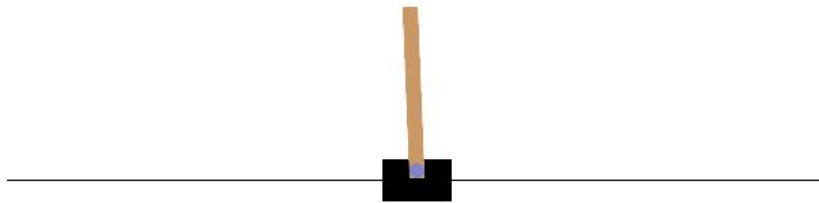
CartPole 환경

- 상태: 4차원
 - 수레 위치 ($-4.8 \sim +4.8$)
 - 수레 속도 ($-\infty \sim +\infty$)
 - 막대 각도 ($-0.42 \sim +0.42$ rad)
 - 막대 각속도 ($-\infty \sim +\infty$)



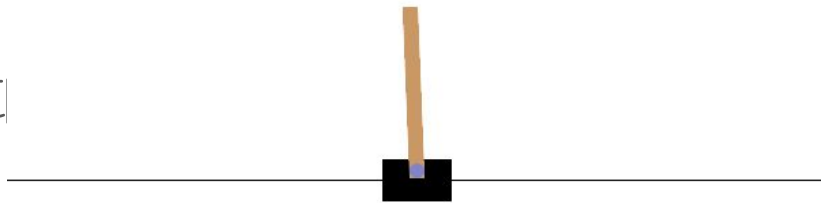
CartPole 환경

- 행동:
 - 왼쪽(0)으로 밀기
 - 오른쪽(1)으로 밀기



CartPole 환경 종료 조건

- 보상: 막대를 최대한 오래 세워두기
- CartPole 에피소드 종료 시점:
 - 막대 각도가 $\pm 12^\circ$ 초과 (실패)
 - 수레 위치가 ± 2.4 범위 초과 (실패)
 - 500 타임스텝 도달 (성공)



Jupyter Notebook (주피터 노트북)

`jupyter notebook`

To access the server, open this file in a browser:

`file:///C:/Users/Lesenic/AppData/Roaming/jupyter/runtime/jpserver-49172-open.html`

Or copy and paste one of these URLs:

`http://localhost:8888/tree?token=7a5fb4cb6ccce0a6cbcd400bda409ce892a7cfe7869dad50`

`http://127.0.0.1:8888/tree?token=7a5fb4cb6ccce0a6cbcd400bda409ce892a7cfe7869dad50`

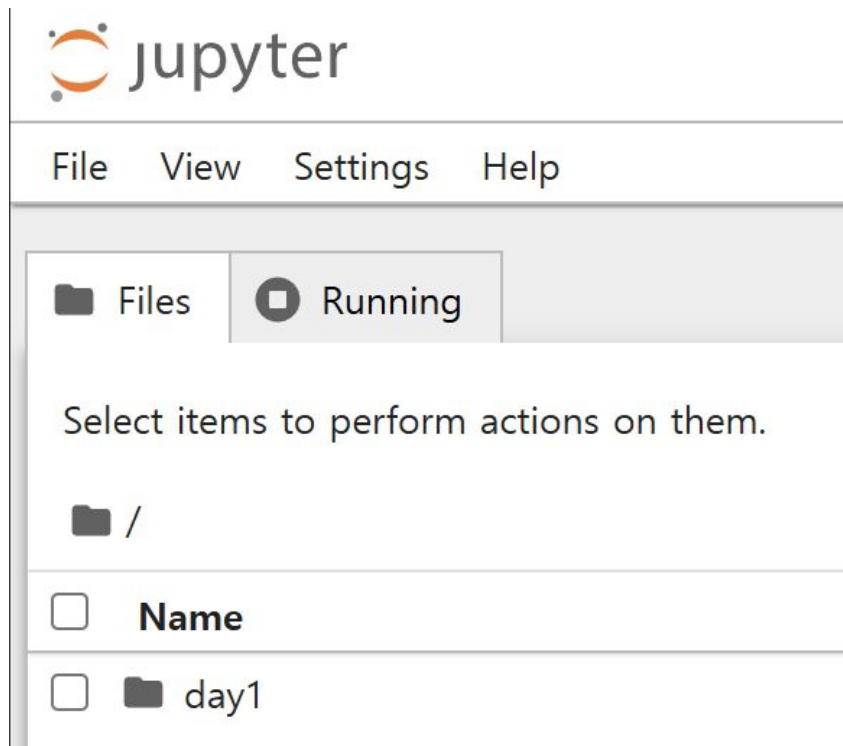


Enter the URL of the running Jupyter Server

`http://localhost:8888/tree?token=7a5fb4cb6ccce0a6cbcd400bda409ce892a7cfe7869dad50`

Press 'Enter' to confirm your input or 'Escape' to cancel

Jupyter Notebook (주피터 노트북)



Jupyter Notebook (주피터 노트북)

- 웹 브라우저 상에서 실행되는 '대화형 컴퓨팅 환경'
- 복잡한 설정 없이 코드 작성부터 실행, 문서화까지 한 곳에서 처리
- 코드를 '셀(Cell)' 단위로 나누어 작성하고 한 줄씩 즉시 실행
- 그래프, 표, 이미지, 마크다운(Markdown) 텍스트를 파일에 통합 가능

상태 → 행동 → 보상 학습 순환

[스텝 49] 행동: 왼쪽 (0)

[0] 카트 위치 : +0.303608

[1] 카트 속도 : +1.342191

[2] 막대 각도 : -0.243255

[3] 막대 각속도 : -1.875915

보상: 1.0, 종료: True

에피소드 종료: 49 스텝, 총 보상 49.0

상태 → 행동 → 보상 학습 순환

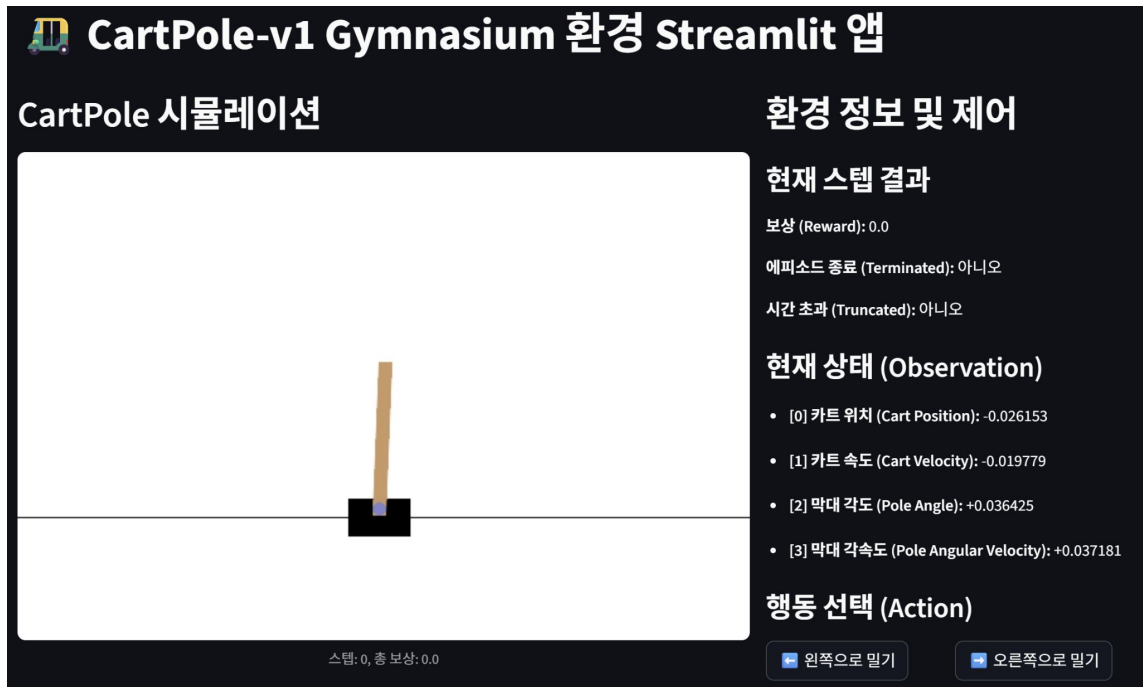


Streamlit (스트림릿)

- HTML, CSS, JavaScript 지식 없이 순수 Python 코드만으로 데이터 기반의 웹 애플리케이션을 빠르게 구축
- 슬라이더, 버튼, 입력창 같은 다양한 UI를 간단한 파이썬 함수 호출(``st.slider`` 등)만으로 구현
- 코드를 수정하면 웹 화면에 즉시 반영되는 '실시간 리로딩' 기능 지원
- 데이터 시각화와 AI 모델 데모를 손쉽게 만들 수 있음
- https://huggingface.co/spaces/Kaludi/Stable-Diffusion-Prompt-Generator_App

Streamlit (스트림릿)

```
streamlit run day1/03_cartpole_streamlit.py
```



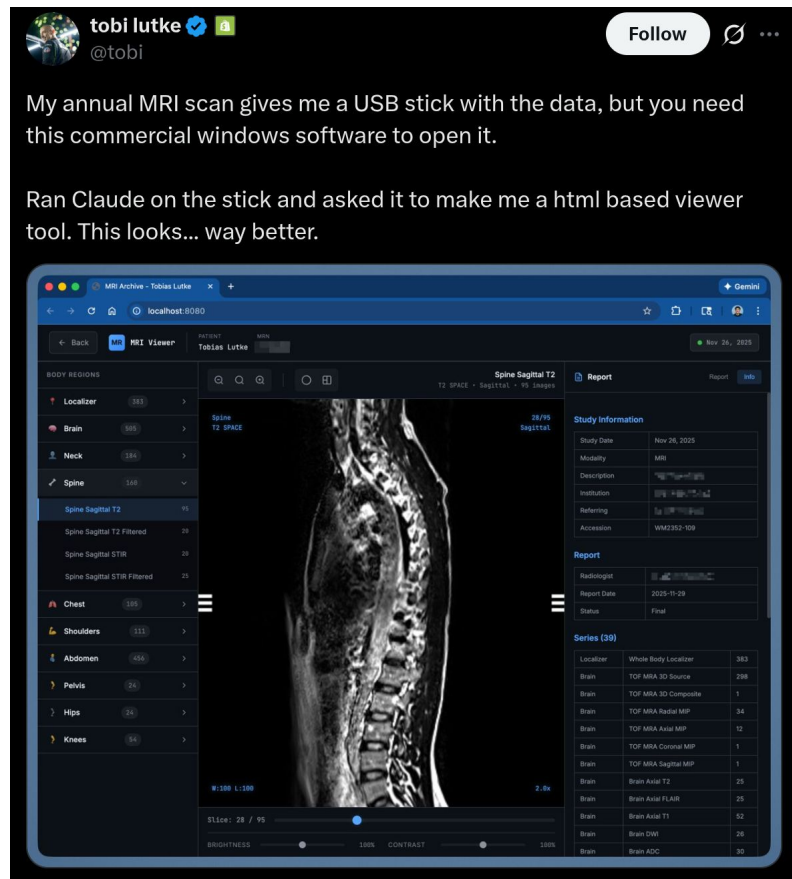
바이브 코딩

- 안드레 카파시 (OpenAI 공동 창업자)가 제창
 - 자연어와 '느낌(Vibe)'으로 AI에게 의도를 자연어로 전달하면 AI가 개발을 진행하는 방식
- 과거 몇 달 걸리던 작업이 이제 몇 시간 안에 가능
- 개발자 소수만 가능
→ 좋은 아이디어만 있으면 누구나 앱 구현 가능



바이브 코딩

- Tobias Lütke (토비아스 뢰트케, Shopify CEO)
- 상용 소프트웨어 수준의 MRI 대시보드를 직접 제작
- 클로드 코드를 통해 제작
- <https://x.com/tobi/status/2010438500609663110>



AI 코드 에디터(개발자 중심)

- [Cursor](#)(커서): VS Code 기반, Tab 자동완성, Composer로 여러 파일 동시 생성/수정, 현황은 AI 코드 에디터의 '표준(Standard)'로 자리잡음
- [Claude Code](#)(클로드 코드): 터미널(CLI) 기반 코딩 에이전트, 코드베이스 이해, 명령어 실행·이슈 분석·코드 수정·PR 작성까지 워크플로우 자동 관리
- [Google Antigravity](#)(구글 안티그래비티): 2025년 11월 출시된 차세대 IDE 컨셉, 'AI 에이전트가 스스로 개발' 지향, 다중 에이전트 병렬 협업, 에디터/터미널/브라우저 완전 제어 및 아티팩트 생성, Gemini 3 Pro 기반

풀스택/프로토타이핑 생성 도구 (비개발자·빠른 제작 중심)

- [Lovable](#)(러버블): 자연어로 기능 설명하면 프론트 UI부터 백엔드 (Supabase 등)까지 풀스택 생성, 실시간 결과 확인 및 채팅으로 수정 가능, 현황은 “미친 퀄리티(Insane quality)” 반응으로 소셜·커뮤니티에서 급부상
- [Replit Agent](#)(레플릿): Replit 웹 IDE 내 자율 에이전트, 간단 프롬프트로 기획·환경설정·패키지 설치·코드 작성·배포까지 전 과정 수행
- [v0](#)(브이제로): Vercel의 UI 생성 도구, 텍스트/이미지 입력으로 React+Tailwind 기반 UI 코드 생성, 최근 UI를 넘어 전체 앱 프로토타입 생성으로 확장

바이브 코딩 맛보기

- 구글 Colab(코랩): <https://colab.research.google.com>
 - 별도의 복잡한 설치 과정 없이 웹 브라우저에서 즉시 파이썬 코드를 작성하고 실행할 수 있는 클라우드 기반의 주피터 노트북(Jupyter Notebook) 환경 제공
 - 데이터 분석과 머신러닝 입문용 도구.
 - 고성능 GPU와 TPU 등 값비싼 컴퓨팅 자원을 무료로 활용
 - Gemini를 활용해 자연어로 코드를 생성하고 즉시 실행 결과를 눈으로 확인하는 '바이브 코딩' 가능

바이브 코딩 맛보기

- 덜 완성된 코드를 완성시켜 봅시다.

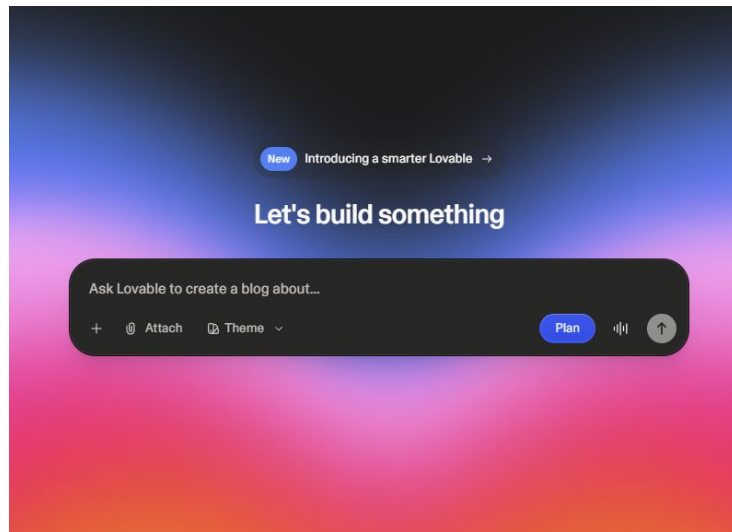
```
# TODO: st.session_state.env.step(action) 호출 후 반환값을 받아
#     st.session_state.state, reward, terminated, truncated,
total_reward, step_count 를 갱신하세요.
```

`day1/04_cartpole_streamlit_colab.ipynb`

`day1/05_cartpole_streamlit_colab_runable.ipynb`

Lovable이란?

- AI와 대화하며 앱과 웹사이트를 만드는 플랫폼
- 채팅으로 원하는 기능을 설명하면 즉시 구현
- 실시간으로 결과를 확인하며 반복 수정 가능
- 프로토타입부터 프로덕션까지 빠른 개발
- <https://lovable.dev>



바이브 코딩

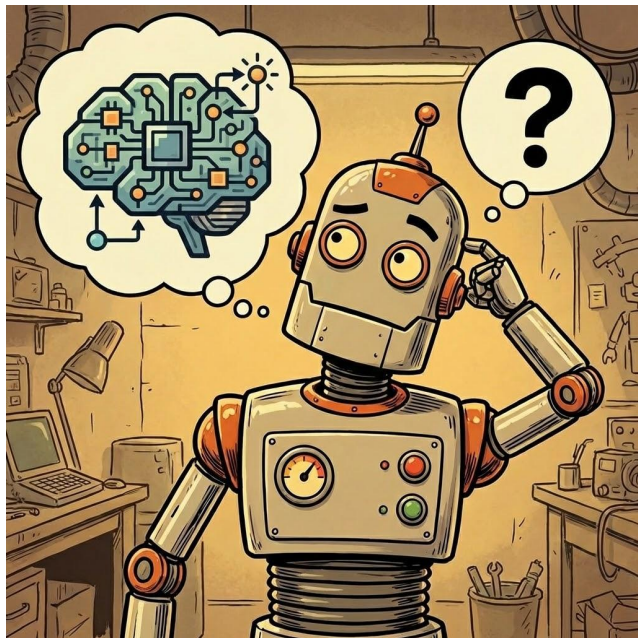
- 클로드 코드로 63일 만에 혼자서 엔터프라이즈급 웹앱 구축한 사연
 - <https://yozm.wishket.com/magazine/detail/3469/>
- 사냥꾼의 행방 (2014-12-28)
 - <https://blog.gorekun.com/?p=5947>

1일차 요약

- 강화학습의 기본 개념 이해
- Gymnasium 환경 사용법 습득
- CartPole 데모 실습
- Colab과 Gemini로 바이브 코딩
- Lovable로 대화를 통해 앱을 개발하는 데모

2일차 예고

- 정책(Policy)의 정의와 종류
- 가치 함수(Value Function) 이해
- 벨만 방정식 소개
- 동적 프로그래밍 기법



`day1/06_cartpole_dqn_demo.ipynb`

감사합니다